

Graph Valid Tree Notes

Problem Statement

You are given n nodes labelled $0, 1, 2, \dots, n-1$ and an undirected edge list `edges`.

Return `true` if the graph forms a valid tree.
Otherwise `false`.

A graph is called a tree iff

- **Condition 1:** There is no cycle.
- **Condition 2:** Every node is connected.

Both conditions must hold.

Examples

Graph

`0 - 1 - 2 - 3`

Can Aadya go from `0` to every node? Yes.

Any cycle? No.

Answer: `true`

Graph

`0`

`/ 1 --- 2`

Can Aadya reach everyone? Yes.

Cycle? `0 → 1 → 2 → 0` Yes.

Answer: `false`

Logic & Shortcuts

A tree is simply: **Connected + No Cycle**

So we need to verify both.

Tree always has exactly `n-1` edges.

Therefore:

Nodes = n, Edges = n-1 shortcut.

If `edges.size() != n-1`, Immediately `false`. No DFS required.

Step-by-Step Approach

1. Check `edges == n-1 ?` No → Return false.
2. Build adjacency list.
3. Run DFS. Need to verify **no cycle** and **everyone visited**.

Why normal DFS is NOT enough?

Suppose `0 — 1`

DFS: `0 ↓ 1`

At node `1`, Neighbour `0` already visited.

Does that mean cycle? No!

That's simply the edge Aadya came from.

Hence we need `parent`.


```

class Solution {
public:
    bool dfs(vector<vector<int>>& adj,
            int currnode,
            int parent,
            vector<int>& seen){

        seen[currnode]=1;

        for(auto &nextnode:adj[currnode]){

            if(!seen[nextnode]){
                if(dfs(adj,nextnode,currnode,seen)){
                    return true;
                }
            }

            else if(nextnode!=parent){ //pehle dekh rkha he aur parent bhi nhi
matlab cycle
                return true;
            }
        }

        return false;
    }

    bool validTree(int n, vector<vector<int>>& edges) {

        //tree hone ka 3 param satya:
        //1) n-1 edges
        //2) ek node se kahi bhi jaa skte
        //3) no cycle

        //tree must have exactly n-1 edges , agar nahi he to tree nahi ho skta
        if(edges.size()!=n-1){
            return false;
        }

        vector<vector<int>> adj(n);

        for(auto &it:edges){
            int u=it[0];
            int v=it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<int> seen(n,0);
    }
};

```

```
// detect cycle:
// needs parent tracking

if(dfs(adj,0,-1,seen)){
    return false;
}

//ensure that everyone was seen atleast once , connected rule
for(int node=0;node<n;node++){
    if(!seen[node]){
        return false;
    }
}

return true;
}
};
```



```

class Solution {
public:

    bool validTree(int n, vector<vector<int>>& edges) {

        //3 basic rules
        //n-1 edges
        //all nodes reachable
        //no cycle

        if(edges.size()!=n-1){
            return false;
        }

        vector<vector<int>> adj(n);

        for(auto &it:edges){
            int u=it[0];
            int v=it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<int> seen(n,0);

        //queue me current node aur parent
        queue<pair<int,int>> q;
        q.push({0,-1});
        seen[0]=1;

        while(!q.empty()){

            int currnode=q.front().first;
            int parent=q.front().second;
            q.pop();

            for(auto &nextnode:adj[currnode]){

                if(!seen[nextnode]){
                    seen[nextnode]=1;
                    q.push({nextnode,currnode});
                }
                //dekh rkha lekin parent nahi hawwww
                else if(nextnode!=parent){
                    return false;
                }
            }
        }
    }
}

```

```
//check if all are reachable
for(int node=0;node<n;node++){
    if(!seen[node]){
        return false;
    }
}

return true;
}
};
```